RAG-Based Customer Support Assistant

TECHNICAL DOCUMENTATION

➢ Introduction

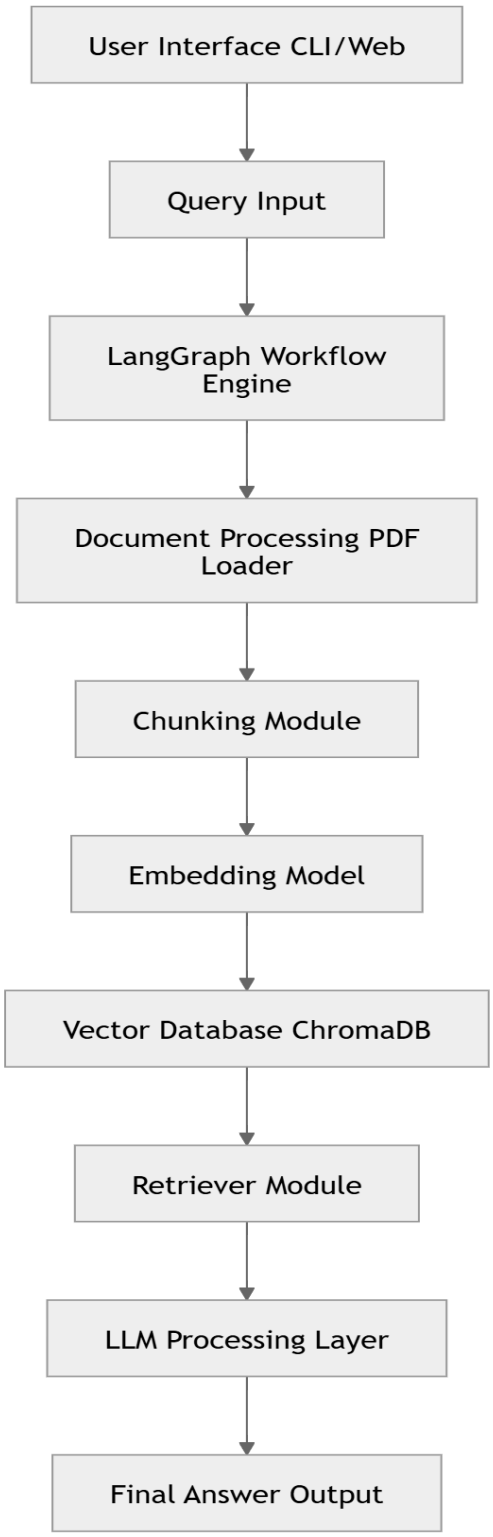
- **What is RAG (Retrieval-Augmented Generation)**
  Retrieval-Augmented Generation (RAG) is an AI framework that combines information retrieval systems with large language models (LLMs). It improves response accuracy by retrieving relevant information from external data sources before generating answers.

- **Why it is needed**
  Traditional LLMs sometimes generate incorrect or hallucinated responses. RAG solves this by grounding answers in real data sources such as PDFs, documents, or databases.

- **Use Case Overview**
  This system is designed as a Customer Support Assistant, where users ask queries (e.g., refund policy, services), and the system retrieves answers from a PDF knowledge base using embeddings and LangGraph workflow.

➢ System Architecture Explanation

- **Architecture Design**



- **Component Interaction**
    1. User submits query
    2. LangGraph manages workflow execution
    3. PDF is processed into chunks
    4. Embeddings are generated
    5. Stored in ChromaDB
    6. Retriever fetches relevant chunks
    7. LLM generates final response
    8. Output is returned to user

➢ Design Decisions

- **Chunk Size Choice**
  Documents are split into small chunks to improve retrieval accuracy and reduce noise. Smaller chunks ensure precise matching of user queries.

- **Embedding Strategy**
  Sentence Transformers are used to convert text into dense vector embeddings that capture semantic meaning rather than keyword matching.

- **Retrieval Approach**
  ChromaDB is used for similarity search using cosine similarity to fetch most relevant chunks.

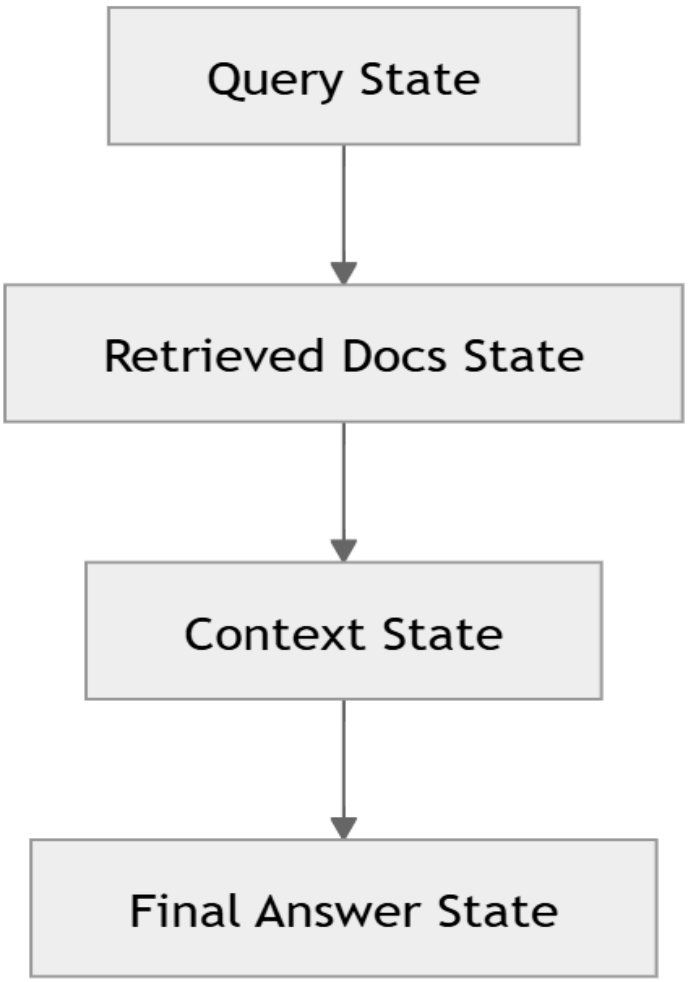
- **Prompt Design Logic**
  Retrieved context is passed to LLM with structured prompt:
  - Context from documents
  - User query

➢ Workflow Explanation

- **LangGraph Usage**
  LangGraph is used to define execution flow between nodes such as input processing, retrieval, and output generation

- **Node Responsibilities**
  Processing Node → Handles query and retrieval
  Output Node → Generates final response

- **State Transistion**



➢ Conditional Logic

- **Intent Detection**
  System checks query intent using keyword matching and context relevance.

- **Routing Decisions**
  If relevant data found → proceed to LLM
  If no data found → fallback response or HITL

➢ HITL Implementation

- **Role of Human Intervention**
  Human-in-the-loop is used when the system cannot confidently answer a query.

- **Benefits**
  - Improves accuracy
  - Reduces hallucination
  - Adds manual verification layer

- **Limitations**
  - Slower response time
  - Requires human available

➢ Challenges & Trade-offs

- **Retrieval Accuracy vs Speed**

  More chunks → better accuracy but slower search
- **Chunk Size vs Context Quality**
  1. Small chunks → precise but may lose context
  2. Large chunks → more context but noisy
- **Cost vs Performance**

  LLM calls increase cost but improve quality

➢ Testing Strategy

- **Testing Approach**
  - Unit testing for each module
  - End-to-end testing of full pipeline
  - Query validation testing
- **Sample Queries**

  1.“What is refund policy?”

  2.“What services are provided?”

  3.“Pricing details?”

➢ Future Enhancements

- **Multi-Document Support**
  Extend system to handle multiple PDFs or knowledge sources.

- **Feedback Loop**
  Store user feedback to improve retrieval quality over time.

- **Memory Integration**
  Add conversational memory for context-aware chat.

- **Deployment**
  Deploy system using:
  - FastAPI backend
  - Docker container
  - Cloud hosting (AWS / Render / Azure